Module 2

Simulink, MATLAB's Control System Functions, and Live Editor

In this module, we present a brief introduction to Simulink and review some MATLAB control system functions from its Control System Toolbox used in the engineering approach to reinforcement learning (RL). In addition, we present basics of MATLAB's live functions and live scripts including interactive live tasks, and several examples that will be used in the follow-up modules in some of the RL design projects. We present also several Simulink based exercises and define some homework problems with MATLAB functions relevant to RL, as starting (warmup) steps for the RL design projects presented in this collection of modules. Students and readers who need more comprehensive coverage of the corresponding MATLAB topics are advised to visit the MathWorks website and to take the abbreviated courses "MATLAB Onramp" and "Simulink Onramp", or simply look for additional information about any particular issue of interest on the MathWorks website.

2.1 Brief Introduction to Simulink

Simulink is one of several MATLAB tool boxes. It can be used for analysis, design, and simulation of linear and nonlinear dynamic systems. This is a brief introduction to Simulink and its blocks used to build dynamic system block diagrams. The emphasis in this presentation is on control systems with potential use for RL. The ways block diagrams are built and simulation performed using Simulink are demonstrated on examples of a drone yaw angle control via a classic PID controller, design of linear observers, eigenvalue assignment feedback control, and control of nonlinear systems via the use of a linearization technique.

Simulink can be accessed by typing "simulink" in the MATLAB window, or by clicking the left mouse button on the MATLAB icon bar denoted as "Simulink". When the Simulink window is open click on "Blank Model" using the left mouse button in order to create a desired block diagram. That opens a new window named "untitled" that at the end of the work should be saved under the name you want.

Simulink has several libraries, from which we take (drug) blocks (icons) to build a block diagram. In addition to the libraries described in the following paragraphs, used for analysis and simulation of continuous- and discrete-time dynamic systems, Simulink contains other libraries not essential for linear and nonlinear feedback control systems and engineering approach to RL.

The Continuous Library contains the transfer function, integrator, derivative, state space, zero-pole (transfer function), PID-controller blocks, along with others not very important block for reinforcement learning.

The Discrete Library contains the discrete transfer function, discrete zero-pole (transfer function), unit delay element, discrete state space, zero-order hold, and discrete filter blocks, among others.

The Math (Mathematics) Library contains the sum, min, max, product, dot product, gain, math function, trigonometric function, and absolute value blocks, among many others.

The Source Library contains the clock (including a digital clock), constant, step, ramp, sine wave, pulse generator, signal generator, from file and from workspace blocks, band-limited white noise generator (used for the Kalman filter), among other blocks of sources representing input signals.

The Sink Library contains the scope, display, to file and to workspace blocks.

The Signal Routing Library contains the mux (signal multiplexer) and demux (signal demultiplexer) needed for the design of linear full- and reduced-order observers.

These libraries are accessed by clicking the left mouse button on the Library Browser icon.

Drawing a block diagram using blocks from the libraries is simple, and is based on several very simple rules given below.

Basic Rules for Building Block Diagrams

A library is opened by clicking the *left mouse button once* on the given library. All icons in the given library will be displayed. We can drag any icon (block) to the working window by pointing the arrow at the icon, and *holding the left mouse button while dragging* the icon to the desired position in the working window. After the mouse button is released, the icon will stay stationary at the given position. After collecting all blocks and positioning them in the working window according to the structure of the block diagram, we must connect them by lines (to indicate the flow of signals). The lines are drawn using the left mouse button: the arrow is positioned at the output of a block, and (holding the left mouse button) the line is drawn to the input of the next block. To start a new line at a point on an existing line, the arrow is brought to that point and the right mouse button is used to start the new line. By releasing the right mouse button, the new line will be terminated, and it can be continued to any other block using the left mouse button as just explained.

Every block in the working window can be opened by double clicking on it with the left mouse button. When the block is opened, a new set of parameters can be assigned. Any block in the block diagram can be enlarged by pointing the arrow at the corner of that block and moving the mouse while holding the left mouse button. System models built in Simulink have an extension `mdl`. For example, using the model name as "drone", the Simulink block diagram is named `drone.mdl`.

Saving, Printing, Inserting (Copying) Block Diagrams Into Documents

Use "Save" and "Save as" to save the block diagram under given name and place it in the desired file. In order to print a Simulink block diagram and/or to make it ready for inclusion in a Microsoft Word Document, use "print" command. For example, if we want the "drone" block diagram to be printed in the "jpeg" format and named it as figure1, use the following MATLAB statement: `>> print -djpeg -sdrone figure1.jpeg`. This instruction creates the file named "figure1.jpeg". In the Microsoft Word document use "Insert" to insert "figure1.jpeg" from a given file. Similarly, one can create other figure formats like "bmp", "tif", etc. Some other

options are also available for saving, printing, copying, inserting Simulink generated block diagrams.

Plotting Signals

Arbitrary number of scopes may be placed in the block diagram to monitor signals. By double clicking on the "Scope" the signal will be displayed. The signals and time may be passed to the MATLAB working window via the "to workspace block", where they can be plotted using `>>print(out.time,out.signal)`. Such obtained signal plots may be farther edited using "grid", "xlabel", "ylabel", and other figure editing MATLAB capabilities, and saved in the desired figure format. For plotting discrete-time signals, the MATLAB function `stairs` is used.

Simulation

When all block parameters are set, simulation can be started by clicking on "Simulation" and then on the green circle "Run" (again with the left mouse button). The simulation time interval set by default to 10.0 seconds can be changed as necessary. Several other simulation options are available on the simulation icon bar.

2.2 Simulink Exercises (Demos)

In this section, we present four exercises (demos) as a good warmup practice for the use of Simulink in diverse RL problems for engineering applications. To that end, we consider the following exercises: a proportional-integral-derivative (PID) controller for a drone, eigenvalue assignment feedback controller, linear observer design, and control of a nonlinear system (simple pendulum) via linearization.

Exercise 2.1: A Drone Yaw Angle Controlled by PID Controllers

In the master thesis [1], the transfer functions of the linearized drone model are derived for three basic angles that define drone's motion: pitch, roll, and yaw. The yaw angle transfer function is given by

$$G_{yaw}(s) = \frac{750}{s(s+0.7)(s+30)} = \frac{750}{s^3 + 30.7s^2 + 21s}$$

The goal is to control the yaw angle subjected to the unit step input using the PID controller, with the controller coefficients selected using the classic *Ziegler-Nichols formulas*:

$$G_{PID}(s) = K_p + K_d s + K_i \frac{1}{s}$$

$$K_p = 0.6 \times Gm\,, \qquad K_i = K_p \frac{\omega_{cp}}{\pi}, \qquad K_d = K_p \frac{\pi}{4\omega_{cp}}$$

where Gm represents the gain margin and ω_{cp} is the phase crossover frequency. For the example

$$Gm = 0.8596, \quad \omega_{cp} = 4.5826 \frac{\text{rad}}{\text{s}}, \quad K_p = 0.5158, \quad K_i = 0.7523, \quad K_d = 0.0884$$

The gain margin is also called the *ultimate gain* K_a, and the phase crossover frequency is called the *ultimate frequency* ω_u. Note that K_u and ω_u can be found either from the Nyquist plot or Bode diagrams or by using the Routh-Hurwitz stability test (and check the gain value for which the system stability is lost, when the corresponding poles on the imaginary axis are $\pm j\omega_u$). These parameters can be found using MATLAB as

```
>>[Gm,Pm,wcp,wcg]=margin(num,den).
```

The PID controller is still widely used in practice. It is also called the industrial controller. Many variants of the PID controllers have been derived since its initial invention. See references [2]-[4], including two books [3]-[4] published recently on the design of PID controllers.

We can call the original PID controller the *plane PID controller*. In general, the pure derivative is difficult to implement in practice. Moreover, the pure derivative is known to amplify noise. In practice, the PID controller is implemented as the *PID controller with the approximate derivative* using the following form

$$G_{PID}(s) = K_p + K_d \frac{s}{1+\mu s} + K_i \frac{1}{s}, \ \mu \text{ is a small positive number } (10^{-3} \text{ to } 10^{-6})$$

Simulink has a built-in PID block that "includes additional features such as anti-windup, external reset and signal tracking". This "advanced" PID controller requires only that the gains K_p, K_i, K_d be specified.

In order to build this block diagram, we must get the "transfer function block" and the "PID controller block" from the Continuous Library, "step and clock blocks" from the Sources Library, "sum block" from the Math Library, and "scope" and "to workspace" blocks from the Sinks Library. The "to workspace block" passes the data from Simulink to MATLAB. Be sure that these signals are defined as "Array", which can be achieved by clicking on the "to workspace block" and selecting the signal format as "Array". First, we drag all of the required blocks from the given libraries into the working window and position them as desired. Then, we connect them by drawing the lines among the blocks, as explained previously in the paragraph on Basic Rules for Building Block Diagrams.

The next step is to setup parameters. For example, we double click on the step block with the left mouse button, and change the step start time from 1 (the default value) to 0, and confirm the change by clicking on "OK." The "sum block" signs should be set to "+-", and the transfer function block parameters should be changed to the system transfer function parameters, equal to "[]" (no real zero in the numerator) for the zeros and [0 -0.7 -30] for the poles, and 750 for the gain. We use the zero-pole version of the transfer function. Available in Simulink is also the numerator/denominator form of the transfer function, in which case the coefficients of the numerator and denominator have to be entered (with the gain included in the numerator coefficients) as [750], [1 30.7 21 0].

Since the results obtained are passed to MATLAB via the "workspace blocks" the yaw angle signal can be plotted by using the following MATLAB functions (see Figure 2.1):

```
>> plot(out.time,out.yaw_angle);
>> grid;
>> xlabel('Time [s]');
>> ylabel('Yaw Angle [deg]')
```

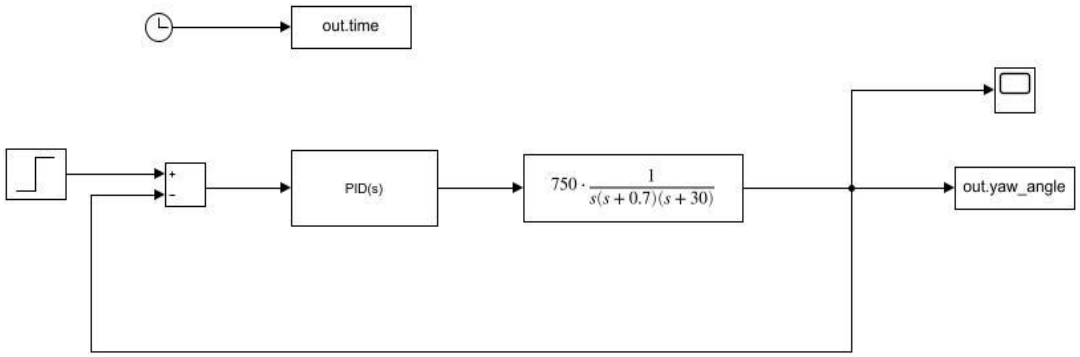


Figure 2.1: Simulink block diagram for a drone yaw angle feedback control using the Simulink built in advanced PID controller

In Figure 2.1, the Simulink built in "PID block" was used, which only requires that the values of gains K_p, K_i, K_d be provided. This can be done by double clicking on the "PID block" and entering these values. However, as stated this PID controller has some enhanced features and it is supposed to be more powerful than the plane PID controller (including its version with the approximate derivative). The corresponding step response is presented in Figure 2.2. It can be observed from Figure 2.2 that the overshoot is around 50%.

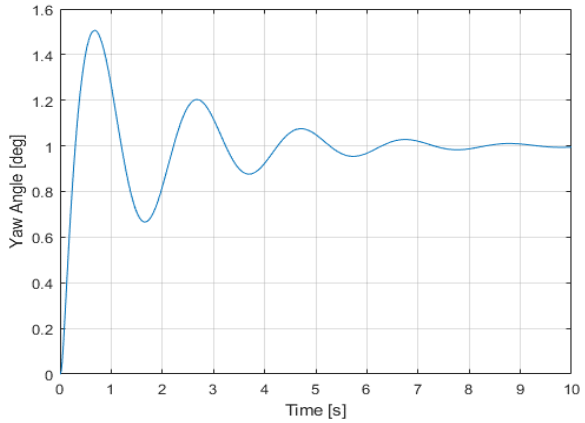


Figure 2.2: Time evolution of the yaw angle using Simulink built-in PID controller whose coefficients are determined using the Ziegler-Nichols formulas

The plane PID controller is built using the corresponding Simulink blocks, and implemented with an approximate derivative $s \approx s/(1+0.000s)$. It is presented in Figure 2.3. It produces only a slightly worse step response comparing the step response of Figure 2.2, presented in Figure 2.4.

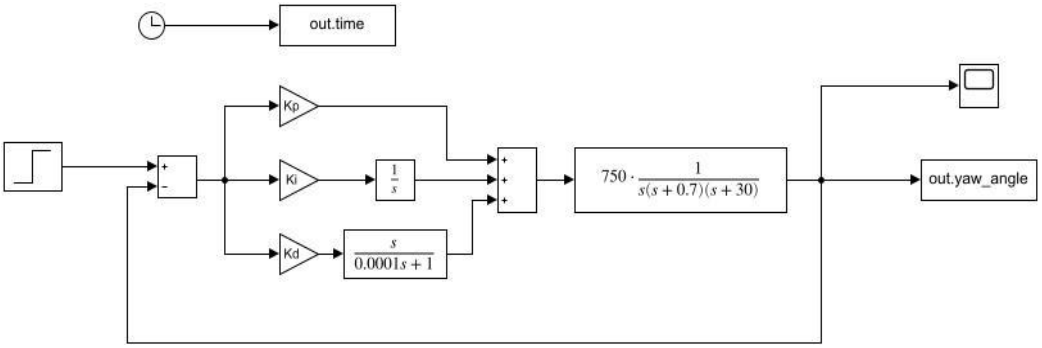


Figure 2.3: PID controller with an approximate derivative for a drone yaw angle control

It is interesting to observe that if we use the root locus technique to design a PD controller and then a PI controller for the same system and then place them in cascade to obtain a PID controller, using the design methodology presented in [5], we got a much better response than the one presented in Figure 2.2, see Figure 2.5. Figure 2.5 shows considerably improved overshoot of 21%, comparing to the 50% overshoot obtained with the MATLAB built in (advanced) PID controller.

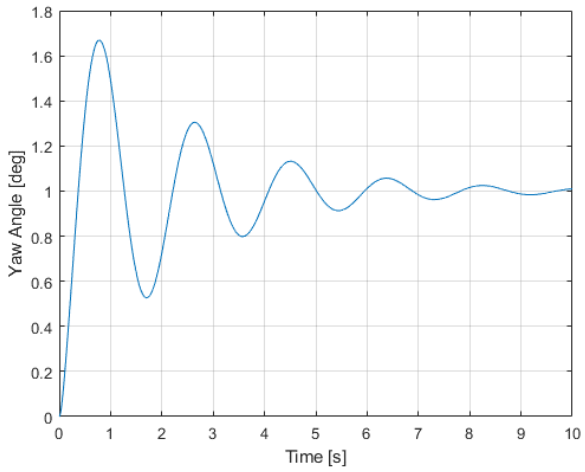


Figure 2.4: Time evolution of the yaw angle under the plane PID controller whose coefficients are determined using the Ziegler-Nichols formulas

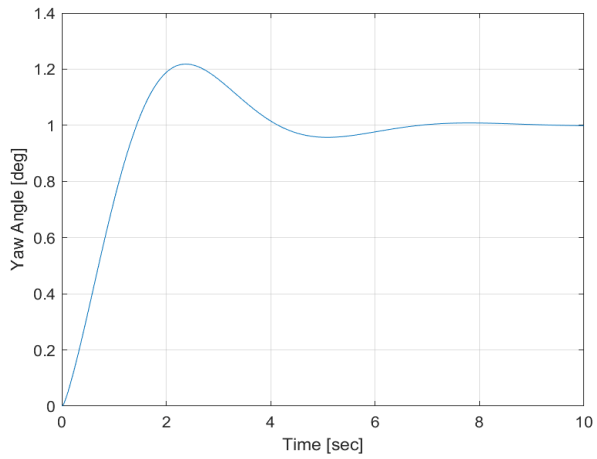


Figure 2.5: Time evolution of the yaw angle under the PID controller designed via the root locus method

Exercise 2.2: A State Space Eigenvalue Assignment Controller

In this exercise, we design the eigenvalue assignment controller (known also as the pole placement controller) using MATLAB and Simulink for the following linear system represented in its state space form by

$$\frac{dx(t)}{dt} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & -2 & -3 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} u(t), \qquad x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$y(t) = x(t)$$

First, we run a MATLAB program to generate the data to be passed to Simulink

```
>> A=[0 1 0; 0 0 1;-1 -2 -3];
>> B=[0;0;1];
>> C=eye(3) %identity matrix; y(t) = C*x(t) = x(t);
>> D=zeros(3,1) % or D=[0;0;0];
>> x0=[1;1;1];
>> lambda_system=[-1+j*1 -1-j*1 -4]
>> % selected desired closed-loop eigenvalues,
>> % the eigenvalues of A-B*F
>> F=place(A,B,lambda_system);
% the required closed-loop feedback gain
```

Then, we built the corresponding Simulink block diagram using the state space block, see Figure 2.6. While building this block diagram, the state space block is taken from the Continuous Library. The "out.time" and "out.y" are to workspace blocks taken from the Sinks Library with the default

name “simout” changed into “time” and “y”. That has been done by using the left mouse button, clicking on these blocks and changing the default name. Also, *very importantly*, while changing the name of the variables, the format of these output variables must be changed into “Array”. The clock is taken from the Sources Library, and the scope is taken from the Sinks Library.

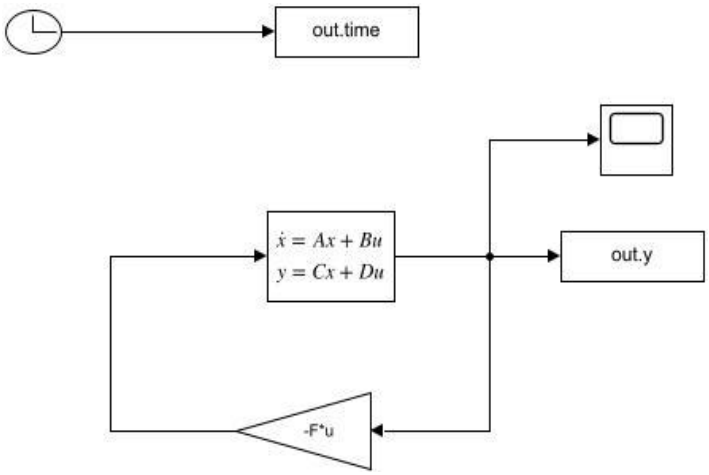


Figure 2.6: Simulink block diagram for the eigenvalue assignment controller $u(x(t)) = -Fx(t)$

The gain block is taken from the Math Library. Once we drag it to the Simulink window, we first have to position it and rotate or flip it such that it is aligned with the flow of signals, which is achieved by clicking the *right* mouse button on the gain block and performing appropriate rotation or flip. By clicking the *left* mouse button on the block, the block will be opened and we will set the gain as $-F$ (feedback gain is always negative). In addition, *we must specify the type of the multiplication* of the matrix gain F and the incoming signal that is considered as a vector. Among four possible choices, we choose “K*u”, which stands for matrix multiplication, more precisely a matrix multiplied by a vector. That will be indicated by “–F*u” appearing in the gain block, which in fact corresponds to actual multiplication $-Fx(t)$. The gain block should be appropriately enlarged as explained earlier in this document. Note that in the case of Simulink scalar gains (which will be used in the next demo), the type of multiplication is “element-wise”, whose notation is “K.*u”.

By clicking on the scope with the left mouse button, all variables in $y(t)$ (note $y(t) = x(t)$) will be presented, but the picture is of a poor quality. It is better to go to the MATLAB window and use the command `>>plot(out.time,out.y)`, which will produce a clear picture that can be edited, for example, by adding the grid and labeling the axis. Such a picture is presented in Figure 2.7.

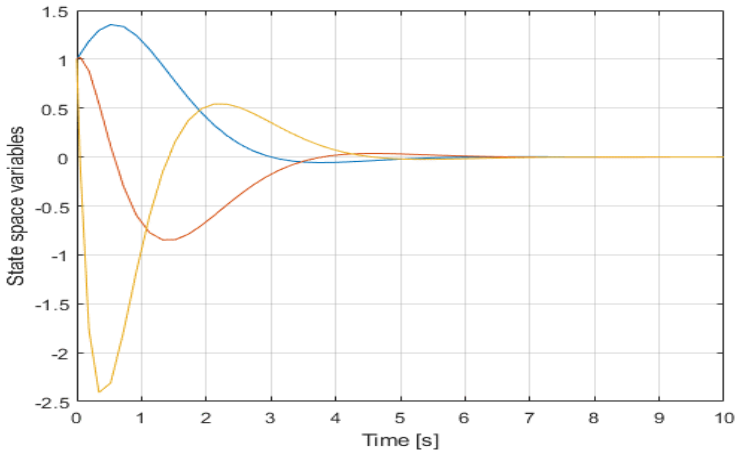


Figure 2.7: The response of the state space variables for the considered example

Exercise 2.3: Design of a Full-Order Observer

The state space blocks are also used in the design of observers. Since the Simulink state space block has only one vector input and one vector output, the two input signals coming into the observer have to be first multiplexed and replaced by one vector signal whose dimensions are equal to the sum of the dimensions of the two vector signals. The full-order observer is given by [6].

$$\frac{d\hat{x}(t)}{dt} = (A - KC)\hat{x}(t) + Bu(t) + Ky(t) = (A - KC)\hat{x}(t) + \begin{bmatrix} B & K \end{bmatrix} \begin{bmatrix} u(t) \\ y(t) \end{bmatrix}$$

The system-observer configuration is presented in Figure 2.8. The observer input matrix must be set as $[B\ K]$. Using $[K\ B]$ will cause an error since the signals are multiplexed by $u(t)$ on the top and $y(t)$ on the bottom. For more details regarding MATLAB/Simulink implementation of observers, the reader is referred to control system education papers [7]-[8]. It follows that the observer state space block should have the state matrix as $A - KC$, the input matrix as $\begin{bmatrix} B & K \end{bmatrix}$, and with both observer block output matrices C and D being zeros of appropriate dimensions.

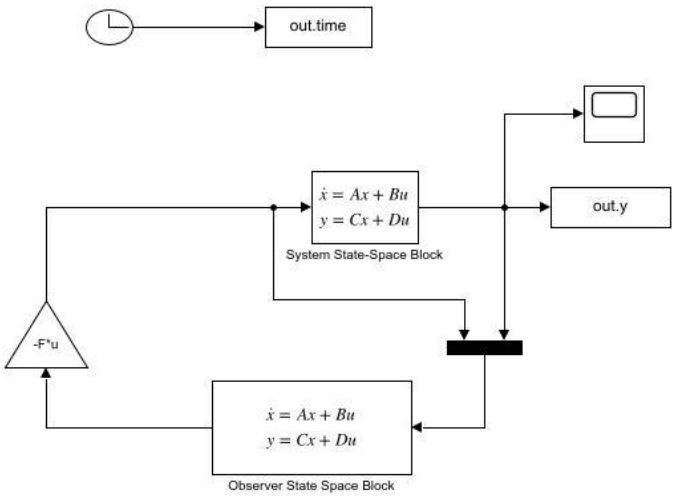


Figure 2.8: Design of an observer with multiplexing of the observer input signals

Exercise 2.4: Control of a Nonlinear System via Linearization
Consider a nonlinear mathematical model of a simple pendulum given by

$$\frac{d^2\theta(t)}{dt^2} + \alpha \frac{d\theta(t)}{dt} + \Omega^2 \sin\left(\theta(t)\right) = \beta f(t)$$

$\theta(t)$ is the pendulum angle with respect to the vertical line, $f(t)$ is the force acting on the pendulum, and α, β, Ω^2 are known constants, $\alpha = 5, \beta = 1, \Omega^2 = 64$. Using the standard change of system variables as $x_1(t) = \theta(t), x_2(t) = \dot{\theta}(t), u(t) = f(t)$, the simple pendulum state space form is obtained as

$$\frac{dx_1(t)}{dt} = x_2(t)$$
$$\frac{dx_2(t)}{dt} = -\alpha x_2(t) - \Omega^2 \sin\left(x_1(t)\right) + \beta u(t)$$

The output of this system is the simple pendulum angle $\theta(t)$, so that the system output equation is given by $y(t) = \theta(t) = x_1(t)$. The goal is to design a feedback controller to keep the steady state value of the simple pendulum angle at $\theta_{ss} = 30^\circ = \pi/6$ [rad]. The simple pendulum steady state values are obtained from the following system of algebraic equations, obtained by setting both derivatives to zero

$$\frac{dx_{1ss}}{dt} = 0 = x_{2ss} \quad \Rightarrow \quad x_{2ss} = 0$$

$$\frac{dx_2(t)}{dt} = 0 = -\alpha x_{2ss} - \Omega^2 \sin\left(x_{1ss}\right) + \beta u_{ss}$$

$$\Rightarrow \quad u_{ss} = \frac{1}{\beta}\left(\alpha x_{2ss} + \Omega^2 \sin\left(x_{1ss}\right)\right) = 64 \sin\left(\theta_{ss_desired}\right)$$

$$y_{ss} = x_{1ss} = \theta_{ss} = \theta_{ss_desired} = 30^\circ$$

Using the linearization method, [6], [9], the linearized pendulum model is obtained as

$$\frac{d\Delta x(t)}{dt} = A\Delta x(t) + B\Delta u(t) = \begin{bmatrix} 0 & 1 \\ -\Omega^2 \cos(x_{1ss}) & -\alpha \end{bmatrix} \Delta x(t) + \begin{bmatrix} 0 \\ \beta \end{bmatrix} \Delta u(t)$$

$$= \begin{bmatrix} 0 & 1 \\ -64\cos(\pi/6) & -5 \end{bmatrix} \Delta x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Delta u(t)$$

with

$$x(t) = x_{ss} + \Delta x(t), \quad u(t) = u_{ss} + \Delta u(t)$$

The feedback controller $\Delta u(\Delta x(t)) = -F\Delta x(t)$ is selected using the eigenvalue assignment procedure via the following MATLAB program

```
>> alpha=5; beta=1;omega=8
>> x1ss=pi/6; x2ss=0; uss=omega^2*sin(x1ss);
>> % uss is known as the set-point controller
>> A=[0 1; -omega^2*cos(x1ss) -alpha]
>> B=[0;beta]
>> lambda_desired=[-5+j*5 -5-j*5]
>> F=place (A,B,lambda_desired)
>> y_desired=x1ss
```

The corresponding Simulink block diagram is presented in Figure 2.9. The integrator blocks ($1/s$) are dragged from the Continuous Library. Note that if the system model built using integrators has non-zero initial conditions, those values can be entered by clicking the left mouse button on the integrator blocks and appropriately setting the initial conditions. In this example, the first integrator contains information about $x_2(0)$ and the second integrator about $x_1(0)$. The default values in the integrators for system initial conditions are set to zeros, which happens to be the case in the considered example. The blocks that define the constant input values for "y_desired" and "x2ss" are taken from the Sources Library. The "sin" function block can be found in the Math Library.

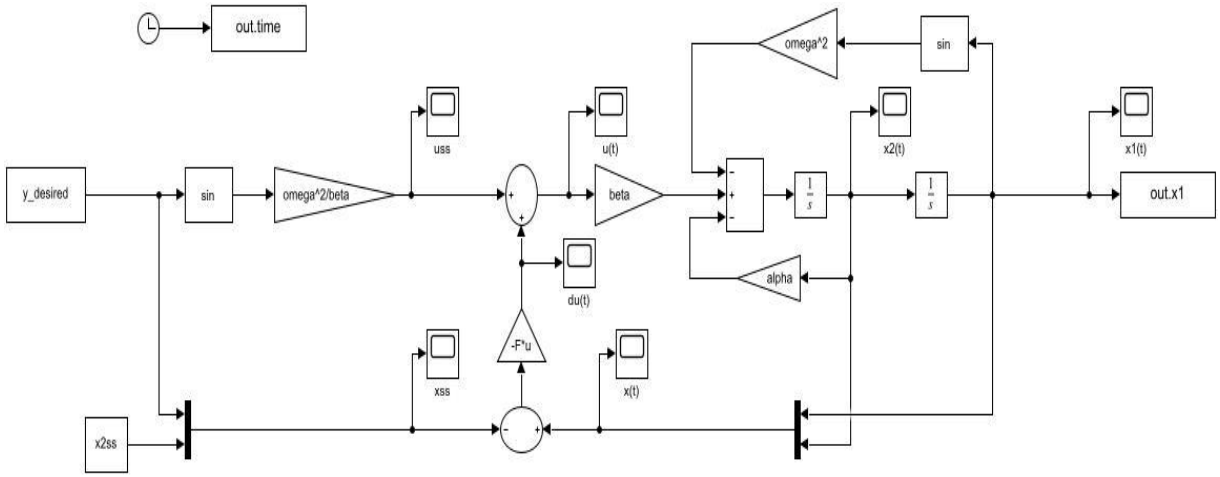


Figure 2.9: Simulink block diagram for simple pendulum control via linearization

After the Simulink block diagram is built, it can be simplified in the sense that some parts of it, for example the system model part, can be represented as a subsystem with given inputs and outputs, which provides more clear picture about the flow and connections of signals. This is simply done by encircling the blocks whose subsystem has to be formed using the left mouse button, and clicking on three dots that appear in the subsystem near the lower right corner. The simplified block diagram is presented in Figure 2.10.

The obtained system output response $\theta(t) = x_1(t) = y(t)$ is presented in Figure 2.11. It can be seen that the response quickly settles down to $\pi / 6 = 0.52$ [rad] with a very small overshoot and short settling time.

Comment 2.1: In addition to the theoretical approach to linearization of nonlinear systems around desired operating (nominal, steady state) points as presented in this exercise by following [6], [9], linearization could be performed using MATLAB function `linmod`, pendulum nonlinear model Simulink block diagram, saved for example as `pendulum.mdl`, and specifying the desired operating points (in this case steady state points) as `>> xss=[x1ss; x2ss]` and `uss` as follows

```
>> [A,B,C,D]=linmod(pendulum.mdl,xss,uss)
```

Similarly, the discrete-time state space model can be obtained via linearization of a continuous-time system whose mathematical model is built in Simulink by specifying in addition the sampling rate "Ts" using the MATLAB command

```
>> [Ad,Bd,Cd,Dd]=linmod(pendulum.mdl,xss,uss,Ts)
```

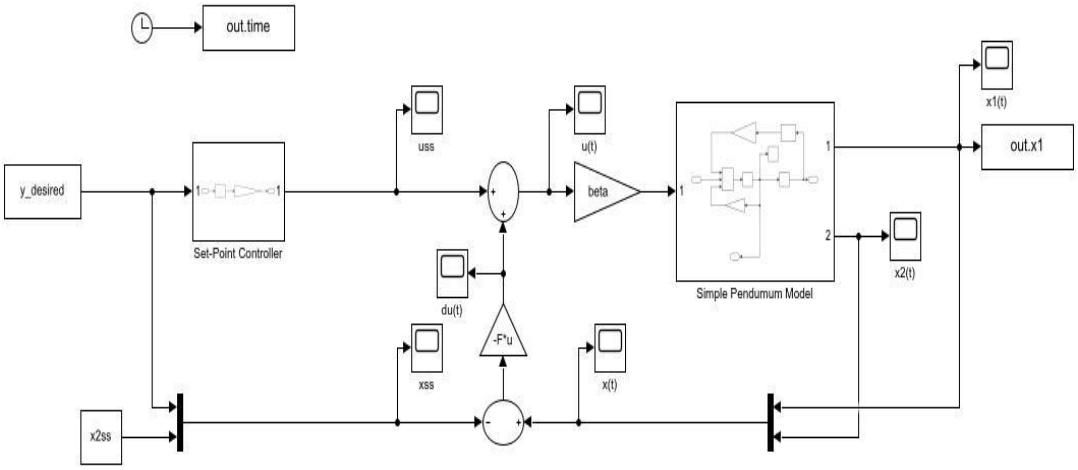


Figure 2.10: Simulink block diagram for the simple pendulum via linearization with subsystems for the set-point controller and for the simple pendulum model

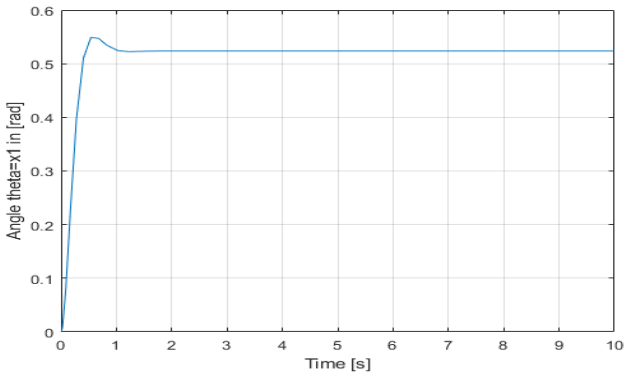


Figure 2.11: The simple pendulum output (angle $\theta(t) = x_1(t) = y(t)$) response

2.3 Some Control System MATLAB Functions Important for RL

In RL for engineering, we often encounter problems that require solving algebraic Lyapunov and Riccati equations in both continuous- and discrete-time domains, which are related to stability, controllability, and optimality of linear dynamic systems. The algebraic Riccati equations are related to optimality, particularly linear-quadratic (LQ) optimality for linear systems and quadratic performance criteria, which includes also the problems of signal optimal filtering from Gaussian white noise (Kalman filter), and H_2-optimization. The corresponding MATLAB functions are: `lyap, dlyap, lqr, dlqr, are`. Note that the discrete-time equivalent of are function known as dare is not any more recommended for use in the most recent versions of MATLAB, instead the use of `idare` is suggested. Moreover, to test whether the required unique solutions of the algebraic Lyapunov and Riccati equations exist, we need to test the system stability,

controllability, and observability, and use the corresponding MATLAB functions, respectively known as `eig, ctrb, obsv`.

The *algebraic Lyapunov equation in continuous-time* is defined by [6], [10]

$$A^{\mathrm{T}}P + PA + Q = 0, \quad Q = Q^{\mathrm{T}} \tag{2.1}$$

Its unique symmetric solution exists if the matrix A is asymptotically stable (it has all eigenvalue in the left half of the complex plane). This equation can be solved using MATLAB as follows:

```
>> evalues=eig(A);
>> % all eigenvalues of A must lie in the left-half plane
>> P=lyap(A',Q)
```

The *discrete-time algebraic Lyapunov equation* is defined by [6], [10]

$$P = A^{\mathrm{T}}PA + Q, \quad Q = Q^{\mathrm{T}} \tag{2.2}$$

Its unique symmetric solution exists if the matrix A is asymptotically stable in the discrete-time domain (it has all eigenvalues within the unit circle of the complex plane). This equation can be solved using MATLAB as follows:

```
>> eig (A);
>> % all eigenvalues of A must lie inside of the unit circle
>> P=lyap(A',Q)
```

The standard form of the *algebraic Riccati equation in continuous-time* is given by [11]-[12]

$$A^{\mathrm{T}}P + PA + R_1 - PBR_2^{-1}B^{\mathrm{T}}P = 0 \tag{2.3}$$

This equation appears in the process of optimizing a quadratic performance criterion along trajectories of a linear continuous-time dynamic system while solving the dynamic optimization problem defined by

$$\begin{aligned} J &= \min_u \left\{ \frac{1}{2}\int_0^{\infty} \left[x^{\mathrm{T}}(t)R_1x(t) + u^{\mathrm{T}}(t)R_2u(t) \right] dt \right\}, \quad R_1 = R_1^{\mathrm{T}} \geq 0, \quad R_2 = R_2^{\mathrm{T}} > 0 \\ &\frac{dx(t)}{dt} = Ax(t) + Bu(t), \quad x(0) = x_0 \end{aligned} \tag{2.4}$$

where $x(t) \in \mathrm{R}^n$ is a vector of system state variables, $u(t) \in \mathrm{R}^m$ is a vector of system controls, A and B are respectively constant system state matrix and control input matrix of appropriate dimensions, and R_1, R_2 are quadratic performance weighted matrices, respectively for the state and control variables. It should be emphasized that there are several forms of the algebraic Riccati equations in continuous-time that come from various types of dynamic optimization problems. Some of them will be encountered in the presented RL design projects. This algebraic Riccati equation has a unique positive semidefinite solution that stabilizes the closed-loop system (makes

the system closed-loop system matrix $A - BR_2^{-1}B^{\mathrm{T}}P$ asymptotically stable) and provides the optimal linear feedback control and the optimal performance criterion value as

$$u^{opt}(x(t)) = -R_2^{-1}B^{\mathrm{T}}Px(t) = -F^{\mathrm{opt}}x(t), \qquad J^{\mathrm{opt}} = \frac{1}{2}x^{\mathrm{T}}(0)Px(0) \tag{2.5}$$

The required positive semidefinite stabilizing solution of the considered algebraic Riccati equation exists under the following controllability and observability assumption.

Assumption 2.1: The pair (A, B) is controllable and the pair $(A, \mathrm{Chol}(R_1))$ is observable [6], where $\mathrm{Chol}(R_1)$ stands for the Cholesky decomposition of R_1.

The Cholesky decomposition of a positive semidefinite matrix can be found in MATLAB using: `chol(R1)`. It will be seen in the follow-up modules that the *controllability and observability assumptions can be replaced by their weaker versions respectively known as system stabilizability and detectability* [6]. The continuous-time algebraic Riccati equation can be solved in MATLAB and the optimal feedback gain and optimal performance (2.5) found using either the `are` (that gives only the solution matrix P) or `lqr` (that provides both the solution matrix P and the optimal feedback gain F^{opt}) MATLAB functions as follows:

```
>> CM=ctrb(A, B); % forms the controllability matrix
>> rank (CM)
>> CO=obsv(A,chol(R1)); % finds the observability matrix
>> rank (CM)
>> P=are(A,B*inv(R2)*B',Q);
>> [Fopt,P]=lqr(A,B,R1,R2)
>> Jopt = 0.5*x0'*P*x0
```

The standard form of the *algebraic Riccati equation in discrete-time* is [11]-[12]

$$P_d = A_d^{\mathrm{T}}P_dA_d + R_{d1} - A_d^{\mathrm{T}}P_dB_d\left(R_{d2} + B_d^{\mathrm{T}}P_dB_d\right)^{-1}B_d^{\mathrm{T}}P_dA_d \tag{2.6}$$

The required positive semidefinite stabilizing solution of the considered discrete-time algebraic Riccati equation (2.6) exists under the following controllability and observability assumption.

Assumption 2.2: *The pair* (A_d, B_d) *is controllable and the pair* $(A_d, \mathrm{Chol}(R_{d1}))$ *is observable [6], where* $\mathrm{Chol}(R_{d1})$ *stands for the Cholesky decomposition of matrix* R_{d1}.

The algebraic Riccati equation (2.6) appears in the process of optimizing a quadratic performance criterion along trajectories of a linear discrete-time dynamic system

$$\begin{aligned} J_d &= \min_u\left\{\frac{1}{2}\sum_{k=0}^{\infty}\left[x^{\mathrm{T}}(k)R_{d1}x(k) + u^{\mathrm{T}}(k)R_{d2}u(k)\right]\right\},\ R_{d1} = R_{d1}^{\mathrm{T}} \ge 0,\ R_{d2} = R_{d2}^{\mathrm{T}} > 0 \\ & x(k+1) = A_dx(k) + B_du(k), \quad x(0) = x_0 \end{aligned} \tag{2.7}$$

where $x(k) \in \mathrm{R}^n$ is a vector of system state variables, $u(k) \in \mathrm{R}^m$ is a vector of system controls, A_d and B_d are respectively constant system state matrix and control input matrix of appropriate dimensions and R_{d1}, R_{d2} are respectively quadratic performance weighted matrices for the state and control variables. Real physical systems are continuous in nature and represented by continuous-time state space form (2.4). The discrete-time systems are either man-made systems or obtained by performing discretization of continuous-time systems using various discretization techniques.

MATLAB Discretization of Continuous-Time State Space Models

MATLAB has a discretization function `c2d.m` that provides the values for the discrete-time state space matrices Ad, Bd, Cd, Dd assuming the continuous time state space matrices Ac, Bc, Cc, Dc and the sampling period Ts are known. The corresponding MATLAB code is as follows:

```
>> sysc = ss(Ac,Bc,Cc,Dc); % continuous state space
>> % the model is discretized with the sampling period Ts
>> sysd = c2d(sysc,Ts); % discrete-time state space model
>> % extraction of discrete-time state space matrices
>> [Ad,Bd,Cd,Dd] = ssdata(sysd)
```

Note that C and D matrices define the system output variables, respectively in the continuous- and discrete-time domains, given by

$$y(t) = C_c x(t) + D_c u(t), \qquad y(k) = C_d x(k) + D_d u(k) \tag{2.8}$$

with appropriate dimensions $C_c = C_d \in \mathrm{R}^{l \times n}, l \le n$ and $D_c = D_d \in \mathrm{R}^{l \times m}$.

It should be emphasized that there are several forms of the algebraic Riccati equations in discrete-time that come from various types of optimization problems. Some of them will be encountered in the presented RL design projects. This algebraic Riccati equation has a unique positive semidefinite solution under Assumption 2.2 that stabilizes the closed-loop system (makes the system feedback matrix $A - BF^{opt}$ asymptotically stable) and provides analytical formulas for the optimal linear feedback control and the optimal performance value as

$$u^{\mathrm{opt}}(x(k)) = -\left(R_{d2} + B_d^{\mathrm{T}} P_d B_d\right)^{-1} B_d^{\mathrm{T}} P_d A_d x(k) = -F_d^{\mathrm{opt}} x(k), \quad J_d^{\mathrm{opt}} = \frac{1}{2} x_0^{\mathrm{T}} P_d x_0 \tag{2.9}$$

It will be seen in the follow-up modules that the controllability and observability conditions can be replaced by their weaker versions respectively known as system stabilizability and system detectability [6]. The discrete-time algebraic Riccati equation can be solved in MATLAB using either the `idare` (that gives only the solution matrix denoted by P_d) or `dlqr` (that provides both the solution matrix P_d and the value for the optimal feedback gain F_d^{opt}) functions by using the following MATLAB command:

```
>> [Fopt,Pd]=dlqr(Ad,Bd,Rd1,Rd2).
```

2.4 MATLAB Scripts, Live Functions, and Live Scripts

Traditionally, programming in MATLAB has been done by writing a sequence of codes and comments representing MATLAB functions and MATLAB commands. Such sequences of codes and comments are in fact MATLAB programs, also known as *MATLAB scripts* (plain codes). The programs (scripts) are saved as m-files by adding to the program's name a suffix (extension) `.m`. The programs (scripts) are commonly executed in the MATLAB environment by typing their name and pressing the return button. Moreover, in addition to common MATLAB functions, one has a possibility to create new MATLAB functions used for specific applications. In the recent years, MATLAB introduced the so-called live functions and live scripts that have interactive capabilities, which enhance the original MATLAB functions and the original MATLAB scripts. MATLAB live functions and live scripts are very convenient in the case of more complex programs since they can be broken into small programs (subroutines) and provide an interactive mode of operation. Here, we briefly present how MATLAB live functions and MATLAB live scripts can be created and utilized with applications to some problems from optimal feedback control systems that play important roles in the engineering approach to RL. As a matter of fact, the *MATLAB live scripts with their interactive capabilities, in conjunction with Simulink with its modeling and simulation capabilities, can create an excellent, software oriented testbed for verification of reinforcement learning algorithms for engineering systems*, see the block diagram in Figure 1.1 with the corresponding text.

2.4.1 MATLAB Live Functions

Live functions are subprograms (subroutines in other programming languages) that can be created independently to perform specific operations, and called from the main program when needed, accepting input data and passing the obtained results (their outputs) to the main program in real time. Live functions are created in the MATLAB Live Editor environment and have extensions `.mlx`, while the main MATLAB program may have either an extension `.m` or `mlx`.

We present two examples, relevant to RL for engineering, to demonstrate how to form live functions: (1) how to examine system controllability and observability, and how to call them from the main MATLAB program, and (2) how to solve the linear-quadratic optimal feedback controller design problem, and how to pass to the main program the values of the obtained optimal feedback gain and the optimal performance criterion.

MATLAB Live Functions from Existing MATLAB Functions

A MATLAB live function having an `.mlx` extension can be simply generated from existing MATLAB functions with an `.m` extension. To do so from the MATLAB Editor use a right-click on the `name.m` document tab and select "Open `name.m` as Live Script". This will generate the live function `name.mlx`. Once a live function `name.mlx` is created from the existing MATLAB function `name.m`, the original `name.m` function is shadowed by the `name.mlx` function, so that `name.m` function can not be run anymore in MATLAB, unless its name is changed.

2.4.1.1 MATLAB Live Functions for Optimal Feedback Control

In this subsection, we demonstrate creation of several fundamental live functions used in optimal feedback control and subsequently in RL. These are the live functions for system controllability, system observability, and optimal control of linear systems with quadratic performance criteria (rewards).

Example 2.1: Live Function for Testing Controllability and Observability

Write a main program `controllability_and_observability.m` that calls live functions named `controllability.mlx` and `observability.mlx`, which independently examine system controllability and system observability. These live functions are called from the main program using their names, providing information about the corresponding system state space matrices, A and B (required for controllability testing), and matrices A and C (required for observability testing), and specifying quantities needed to be passed back to the main program and the MATLAB working window. *Note that the main program and the live functions that it calls must be placed in the same directory.*

% MAIN PROGRAM named `controllability_and_observability.m`

```
% Examination of controllability and observability
clear all
% input the order of the system being examined
n=2;
disp(['System order n = ', num2str(n)])
% "num2str" is used to display numeric data with text
A=[-1 1;
  0 -2];
B=[0;1];
C=[1 0];
% calling live function that examines controllability
controllability(A,B);
% calling the live function that examines observability
observability(A,C);
```

The live function `controllability.mlx` forms the controllability matrix, examines its rank, and determines whether the system is controllable or not. In addition, it transfers back to the main program the numerical value of the rank of the controllability matrix and the order of the system.

```
% CONTROLLABILITY LIVE FUNCTION named controllability.mlx
function [rankCM,n] = controllability(A,B)
CM=ctrb(A,B);
rankCM=rank(CM);
disp(['Rank of controllability matrix = ',num2str(rankCM)])
sizeA=size(A);
n=sizeA(1);
if(rankCM==n)
   disp(['System is controllable'])
```

```
else
   disp(['System is not controllable'])
end
```

The live function `observability.mlx` forms the observability matrix, examines its rank, and determines whether the system is observable or not. In addition, it transfers back to the main program the numerical value of the rank of the observability matrix and the order of the system.

```
% OBSERVABILITY LIVE FUNCTION named  observability.mlx
function [rankOM,n] = observability(A,C)
OM=obsv(A,C);
rankOM=rank(OM);
  disp(['Rank of observability matrix = ',num2str(rankOM)])
sizeA=size(A);
n=sizeA(1);
if(rankOM==n)
  disp(['System is observable'])
else
  disp(['System is not observable'])
end
```

After running the `controllability_and_observability.m` MATLAB program that calls two live functions, the following results are obtained and displayed in the MATLAB window

```
>> controllability_and_observabiity
System order n = 2
Rank of controllability matrix = 2
System is controllable
Rank of observability matrix = 2
System is observable
```

indicating that the considered system is both controllable and observable since the system controllability and observability matrices have full ranks equal to the system order `n=2`.

Example 2.2: Optimal LQ Controller MATLAB Live Function

A main program named `LQ_problem.m` provides the data needed for solving the continuous-time LQ optimal control problem defined in (2.4), that is, it specifies information about matrices A, B, R_1, R_2 and the system initial condition vector x_0. In addition, it provides information which quantities need to be transferred back by the live function to the main program. The live function, `optimal_LQ.mlx`, first tests whether Assumption 2.1 is satisfied, and if yes, it solves the continuous-time algebraic Riccati equation (2.3) and evaluates the optimal feedback gain and the optimal performance criterion defined in (2.5). These values are passed back to the main MATLAB program and to the MATLAB working window. If Assumption 2.1 is not satisfied, the live function states that the sought solution does not exist. In the follow-up material, *we will*

introduce a weaker condition for the existence of the optimal LQ controller in terms of system stabilizability and detectability [6]. At that point we will build another live function for the existence of the LQ controller.

```
% MAIN PROGRAM: LQ OPTIMAL CONTROLLER named LQ_problem.m
clear all
% LQ Problem for a Simple Pendulum from Exercise 2.4
A=[0 1;-64*cos(pi/6)  -5];
B=[0;1];
R1=eye(2);
R2=1;
x0=[0.1; -0.1];
% Testing Assumption 2.1
[rankCM,n]=controllability(A,B)
CC=chol(R1);
[rankOM,n]=observability(A,CC)
% finding the optimal feedback gain and performance criterion
if(rankOM==n & rankCM==n)
   [Fopt,Jopt]=optimal_LQ(A,B,R1,R2,x0);
else
   disp(['Optimal control does not exist'])
end
```

The live function `optimal_LQ.mlx` solves the algebraic Riccati equation and evaluates the optimal feedback gain and the optimal value of the performance criterion.

```
% OPTIMAL LQ CONTROLLER LIVE FUNCTION named optimal_LQ.mlx
function [Fopt,Jopt] = optimal_LQ(A,B,R1,R2, x0)
% optimal feedback gain and the solution of Riccati equation
[Fopt,P]=lqr(A,B,R1,R2)
% optimal value of the performance criterion
Jopt=0.5*x0'*P*x0
end
```

After running the `LQ_problem.m` MATLAB program that calls three live functions the following results are obtained and displayed in the MATLAB window

```
>> LQ_problem
Rank of controllability matrix = 2
System is controllable
rankCM = 2
n =2
Rank of observability matrix = 2
System is observable
rankOM = 2
n = 2
Fopt = 0.0090  0.1008
P = 5.6323  0.0090
```

```
    0.0090   0.1008
Jopt = 0.0286
>> P
   Unrecognized function or variable 'P'.
>> Jopt
   Jopt = 0.0286
>> Fopt
   Fopt = 0.0090    0.1008
>>
```

It should be emphasized that all variables generated by the live function `optimal_LQ.mlx`: `P`, `Jopt`, and `Fopt` are displayed in the MATLAB working window, but the variable `P` is not active since in calling the corresponding live function, that is, `>>[Fopt,Jopt]=optimal_LQ(A,B,R1,R2,x0),` we did not explicitly say that the numerical value of `P` is needed to be returned back to the main program. Matrix `P` is evaluated using the statement `>>[Fopt,P]=lqr(A,B,R1,R2).` Assuming we set ";" after this statement, that is `>>[Fopt,P]=lqr(A,B,R1,R2);` then the value of `P` will not show up in the MATLAB working window.

2.4.2 MATLAB Live Scripts

MATLAB live scripts are interactive MATLAB programs created in the MATLAB Live Editor environment. In addition to the codes and formatted text, they may contain generated outputs and graphics, equations, hyperlinks, images, and videos. For example, the percent symbol used in standard MATLAB scripts for comments can be replaced by the actual text of the comments by selecting "Text" in the Live Script Editor. The live scripts are created in the MATLAB Live Editor by selecting "new live script". Upon creation of a live script program, the program is given a name with an extension `.mlx`, for example `name.mlx`.

When a MATLAB live script is run, the MATLAB working window is divided into three parts. The live script is in the middle, the results are presented to the right of it, and on the left is the classical (old) MATLAB window. The results can be presented also at the bottom of the MATLAB working window by clicking on the "Output inline" button located at the very end to the right. At the same location is the "Output on right" button that presents the results to the right of the MATLAB live scripts.

MATLAB Live Scripts are very convenient for presentations and lecturing since they provide a lot of flexibility and allow annotations and pictorial presentation of the obtained results (outputs) due to additional interactive possibilities (tasks) such as the use of sliders, spinners, buttons, drop-down lists, and other interactive tasks, whose usage and role can be simply obtained by searching the MathWorks website. Some of the interactive live tasks commonly used in reinforcement learning live scripts are:

Sliders: They are used to interactively change numerical values of some parameters used in the MATLAB live script. A variant of a slider is a **range slider** that interactively selects a range of numerical values that can be taken by some of parameters used in the program.

Spinners: They either increment of decrement the value of a variable, and as such provide possibilities to increase or decrease the number of required iterations for repetitive (iterative) processes.

Buttons: They are used to interactively run MATLAB live scripts, by simply clicking on them.

The use of these interactive live tasks will be demonstrated in numerous MATLAB live scripts presented in the follow-up design modules. They can be simply inserted in the live script by opening "Control" in the Live Script Editor, and selecting the desired task. The MATLAB live scripts will be used in most of presented RL engineering design projects in this collection of design modules. They will be supplemented with various live functions coming from optimal feedback control systems with some of them created in this module.

Creating MATLAB Live Functions from Live Scripts via Refactoring

Live functions can be created from existing live scripts by breaking them into smaller pieces (`.m` or `.mlx` functions) by converting selected parts of codes into functions via operation known as *refactoring*. Simple select a part of the code and in the Live Script Editor click on "Refactor". MATLAB will create the corresponding live function and replace the part of the selected code with a call to the created live function. This MATLAB feature will be demonstrated in Module 4, in RL Design Project Program 4.2.

Exporting MATLAB Live Scripts and Results

After running a MATLAB live script, the script and the results displayed can be exported in several formats, for example, the most common pdf and/or Microsoft Word formats. To do so, on the Live Editor tab select "Export" and choose the format. The corresponding format will be saved in the same file where your original MATLAB live script is located. Moreover, the Live Editor export function can be customized by choosing the document paper size, resolution and format of figures, hiding the code and saving (presenting) only the results. To hide the code, you can either click on the "Hide code button" located to the very end to the right of the live script or go to the View tab and click on "Hide Code" icon. To show the code again, click either on the "Output in Line" button or "Output on Right" button.

Converting MATLAB Live Scripts Into MATLAB Scripts

When an existing MATLAB live script (with an `mlx` extension) is available, it can be converted into the MATLAB script (with an `.m` extension) on the Live Editor tab by selecting Save/Save As, and from the dialog box "MATLAB Code File (UTF – 8) (*.m)".

2.4.3 Interplay of MATLAB Live Scripts and Simulink

It is important to emphasize that Simulink can be run/controlled either from MATLAB scripts or from MATLAB live scripts so that the main block diagram of RL (presented in Figure 1.1) *can be fully implemented and run in the MATLAB environment*. Here, we demonstrate this feature in both continuous- and discrete-time domains.

In *continuous time*, the Simulink model from Exercise 2.1 is used, where control of a drone yaw angle was controlled via a PID controller. In discrete-time, an aircraft example from [13]-[14] is considered. In both cases, the assignment of the system closed-loop eigenvalues problem is used to demonstrate this reinforcement learning *very important MATLAB-Simulink interplay*.

Example 2.3: Calling Simulink from the MATLAB Live Script–Continuous Time

The corresponding MATLAB live script that allows interactive changes of the PID controller parameters (with the use of sliders for the PID controller coefficients), running the Simulink model `drone.mdl`, and plotting the obtained results, is presented in Figure 2.12. Moreover, the Simulink generated outputs are returned to the MATLAB workspace, where they can be used for further processing and calculations. The Simulink model built in Figure 2.1 was named as `drone.mdl`. This model is redrawn here for completeness.

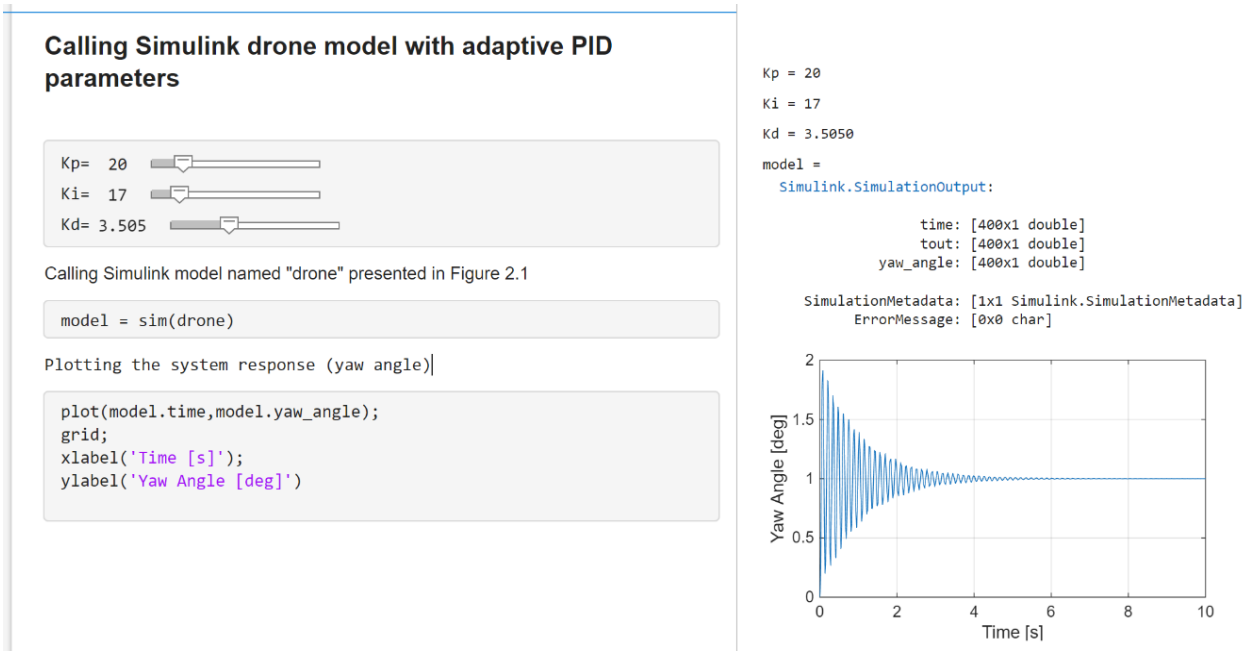
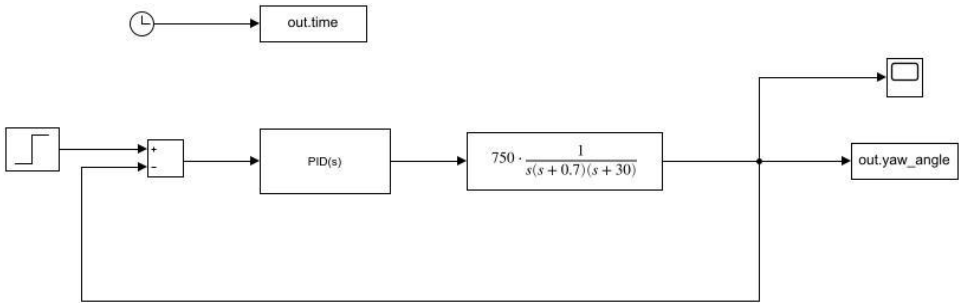


Figure 2.12: The MATLAB live script on the left-hand side, and the obtained results on the right-hand side presented using the snipping tool



Using the Live Editor export function, all data presented in Figure 2.12 could have been generated also in a Microsoft word file or in a pdf file, which is convenient for students when such data are required to be included in a report.

Example 2.4: Calling Simulink from the MATLAB Live Script–Discrete Time

Consider a discrete-time linear system represented in its state space form given by (2.7) with corresponding measurements defined in (2.8). It is named `aircraft.mdl`, since it represent a linearized aircraft model taken from references [13]. The aircraft model is represented in continuous-time in its state space form by

$$\frac{dx_c(t)}{dt} = A_c x_c(t) + B_c u_c(t)$$
$$y_c(t) = C_c x_c(t) + D_c u_c(t)$$

with the following continuous-time state space matrices

$$A_c = \begin{bmatrix} -1.01887 & 0.90506 & -0.00215 & 0 & 0 \\ 0.82225 & -1.07741 & -0.17555 & 0 & 0 \\ 0 & 0 & -20.2 & 0 & 0 \\ 10 & 0 & 0 & -10 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ 0 \\ 20.2 \\ 0 \\ 0 \end{bmatrix}$$

$$C_c = \begin{bmatrix} 0 & 0 & 0 & -57.2958 & 0 \\ 0 & -57.2958 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad D_c = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

This continuous-time model is discretized with the sampling period of $T_s = 0.05\,\text{s}$ using MATLAB function `c2d` via the zero-order hold discretization technique ("zoh" in the MATLAB program) leading to the following discrete-time state space form

$$x_d[k+1] = A_d x_d[k] + B_d u_d[k]$$
$$y_d[k] = C_d x_d[k] + D_d u_d[k]$$

$$A_d = \begin{bmatrix} 0.9512 & 0.0430 & -0.0002 & 0 & 0 \\ 0.0390 & 0.9484 & -0.0054 & 0 & 0 \\ 0 & 0 & 0.3642 & 0 & 0 \\ 0.3829 & 0.0093 & 0.0000 & 0.6065 & 0 \\ -0.0569 & -2.7898 & 0.0091 & 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} -0.0001 \\ -0.0032 \\ 0.6358 \\ 0.0000 \\ 0.0033 \end{bmatrix}$$

$$C_d = C_c, \qquad D_d = D_c$$

The corresponding Simulink block diagram named `aircraft.mdl` for this problem is presented in Figure 2.13.

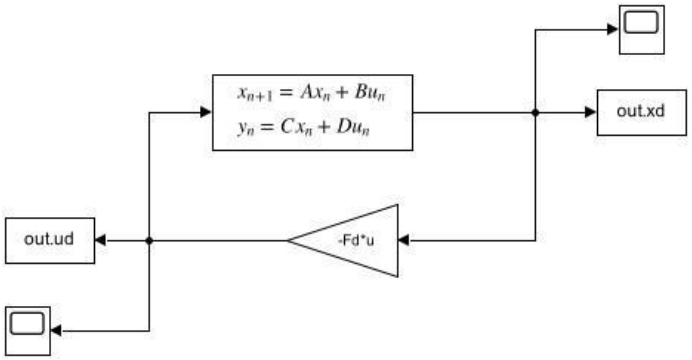


Figure 2.13: Eigenvalue assignment for a discrete-time model of an airplane

We write a MATLAB live script named `evalue_assignment.mlx` that for the desired choices of closed-loop eigenvalues (set interactively using sliders) performs simulation, plots (note the use of MATLAB function stairs for plotting) all state variables, and returns back all state variables to the MATLAB workspace. Using sliders, we are able to change any of the desired eigenvalues. Note that all closed-loop eigenvalues must lie strictly inside of the unit circle to assure system asymptotic stability. By changing any eigenvalue, the MATLAB live script calls the Simulink model `aircraft.mdl`, performs corresponding simulation and plots newly obtained state variables.

Comment 2.2: Instead of defining the time axis as time and evaluating its length in the main live script program in terms of number of sampling periods Ts, one could have used a Simulink digital clock block, as presented in Figure 2.14, and in the live script print the signals directly using the following command (for example, for the state variable $x_{d1}(k)$) as:

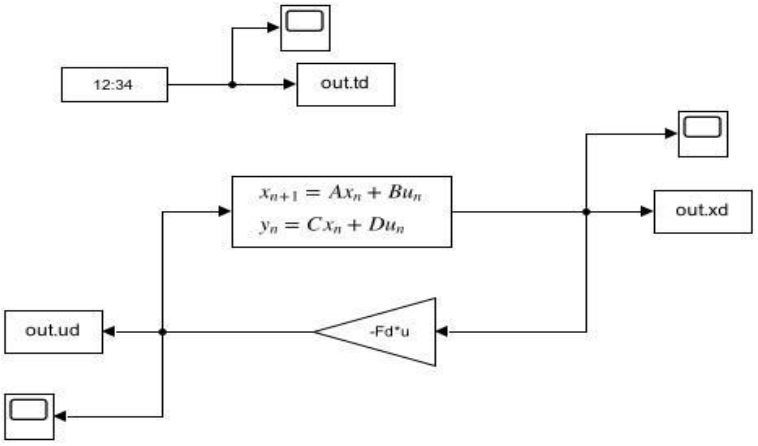


Figure 2.14: Simulink model for an aircraft with the use of a digital clock.

The live script and the results obtained for this example are presented in Figure 2.15.

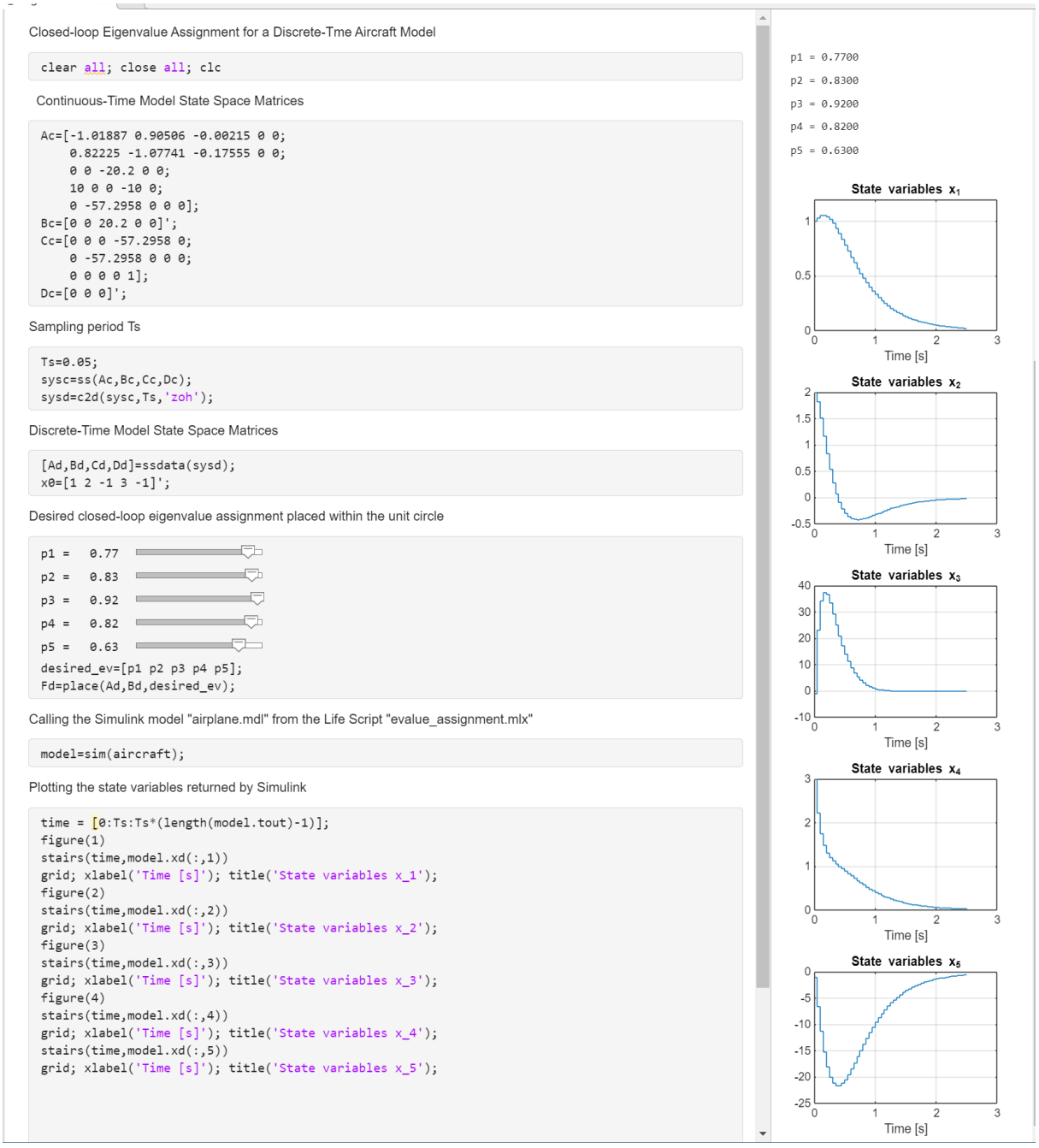


Figure 2.15: Live script and obtained results for a discrete-time eigenvalue assignment problem for an aircraft

Note that in the discrete-time domain, the closed-loop linear system eigenvalues should be all placed strictly inside of the unit circle (with no eigenvalues on the unit circle), that is, $\left|\lambda_i(A_d - B_d F_d)\right| < 1, \forall i$, in order to insure the closed-loop system asymptotic stability. In the continuous-time domain, the linear system asymptotic stability is achieved when all closed-loop eigenvalues are strictly in the left-hand half of the complex plane (with no eigenvalues on the imaginary axis), that is, $\mathrm{Re}\left\{\lambda_i(A_c - B_c F_c)\right\} < 0$.

2.5 References

[1] Reizenstein, A., *Position and Trajectory Control of a Quadcopter [Drone] Using PID and LQ Controllers*, Master Thesis, Linkoping University, Sweden, 2017.

[2] Astrom, K. and T. Hagglund, *Advanced PID Control*, 2nd edition, ISA, 2005.

[3] Diaz-Rodriguez, I., S. Han, and S. Bhattacharyya, *Analytical Design of PID Controllers*, Springer, 2019.

[4] Wang, L., *PID Control System Design and Automatic Tuning using MATLAB /Simulink*, Wiley-IEEE Press, 2020.

[5] Gajic, Z. and M. Lelic, *Modern Control Systems Engineering*, Prentice Hall, London, 1996.

[6] Chen, T-C., *Linear Systems: Theory and Design*, Oxford University Press, Oxford, U.K., 2013.

[7] Radisavljevic-Gajic, V., "Full- and reduced-order linear observer implementation in MATLAB/Simulink," *IEEE Control Systems Magazine*, 91-101, Oct. 2015.

[8] Radisavljevic-Gajic, V., "Linear-quadratic (LQ) optimal steady state controllers for engineering students and practicing engineers," *International Journal of Mechanical Engineering Education*, Vol. 49, 316-358, 2021.

[9] Khalil, H., *Nonlinear Systems*, Prentice Hall, 2002.

[10] Gajic, Z. and M. Qureshi, *Matrix Lyapunov Equation in System Stability and Control,* Academic Press, San Diego 1995. Republished by Dover Publications, Mineola, NY, 2008.

[11] Lancaster, P. and L. Rodman, *Algebraic Riccati Equations*, Oxford University Press, Oxford, U.K., 1995.

[12] Sinha, A., *Linear Systems: Optimal and Robust Control*, Taylor & Francis, 2007.

[13] B. Stevens, F. Lewis, and E. Johnson, *Aircraft Control and Simulation: Dynamics, Control, and Autonomous Systems*, John Wiley and Sons, New York, Third Edition, 2016.